

深圳大学实验报告

课程名称： 算法设计与分析

实验名称： 实验四 动态规划—鸡蛋掉落问题

学院： 计算机与软件学院

专业： 计算机技术与科学

报告人： 姚泽棉 学号： 2024150026 班级： 高性能班

指导教师： 杨烜

实验时间： 2026/05/15——2026/05/24

实验报告提交时间： 2026/05/23

教务处制

一、实验目的

- (1) 掌握动态规划算法设计思想。
- (2) 掌握鸡蛋坠落问题的动态规划解法。

二、问题描述

动态规划将问题划分为更小的子问题，通过子问题的最优解来重构原问题的最优解。动态规划中的子问题的最优解存储在一些数据结构中，这样我们就不必在再次需要时重新处理它们。任何重复调用相同输入的递归解决方案，我们都可以使用动态规划对其进行优化。

鸡蛋掉落问题是理解动态规划如何实现最佳解决方案的一个很好的例子。问题描述如下：



我们需要用鸡蛋确认在多大的楼层鸡蛋落下来会破碎，这个刚刚使鸡蛋破碎的楼层叫门槛层，门槛楼层是鸡蛋开始破碎的楼层，上面所有楼层的鸡蛋也都破了。另外，如果鸡蛋从门槛楼层以下的任何楼层掉落，它都不会破碎。如上图所示，如果有 5 层，我们只有 1 个鸡蛋，要找到门槛层，则必须尝试从每一层一层一层地放下鸡蛋，从第一层到最后一层，如果门槛层是第 k 层，那么鸡蛋就会在第 k 层抛下时破裂，应该做了 k 次试验。也就是说，如果有 k 层楼，1 个鸡蛋，最少的实验次数是 k 次。反之，如果有 e 个鸡蛋，楼层数是 0，则最少试验次数是 0，如果有 e 个鸡蛋，楼层数是 1，最少试验次数是 1。

如果有 6 层楼，三个鸡蛋，需要的最少试验次数是 3；如果有 5 层楼，三个鸡蛋，需要的最少试验次数也是 3。

注意：我们不能随机选择任何楼层，例如，如果我们选择 4 楼并放下鸡蛋并且它打破了，那么它不确定它是否也从 3 楼打破。因此，我们无法找到门槛层，因为鸡蛋一旦破碎，就无法再次使用。

给定建筑物的一定数量的楼层（比如 f 层）和一定数量的鸡蛋（比如 e 鸡蛋），找出阈值地板必须执行的最少的鸡蛋掉落试验的次数，注意，这里要求的是试验的次数，不是鸡蛋的个数。还要记住的一件事是，我们寻找的是找到门槛层所需的最少掉落试验次数，而不是门槛层下限本身。

问题约束条件：

- 从跌落中幸存下来的鸡蛋可以再次使用。
- 破蛋必须丢弃。

- 摔碎对所有鸡蛋的影响都是一样的。
- 如果一个鸡蛋掉在地上摔碎了，那么它从高处掉下来也会摔碎。
- 如果一个鸡蛋在跌落中幸存下来，那么它在较短的跌落中也能完整保留下来。

实验要求：

- 给出解决问题的动态规划方程；
- 随机产生 f, e 的值，对小数据模型利用蛮力法测试算法的正确性；
- 随机产生 f, e 的值，对不同数据规模测试算法效率，并与理论效率进行比对，请提供能处理的数据最大规模，注意要在有限时间内处理完；
- 该算法是否有效率提高的空间？包括空间效率和时间效率。

三、求解问题的算法原理描述

动态规划思想与基础状态定义

设 $dp(e, f)$ 表示在拥有 e 个鸡蛋、 f 层楼时，在最坏情况下确定临界楼层所需的最少实验次数。

若从第 x 层扔下鸡蛋，则会出现两种情况：

鸡蛋破碎：问题规模变为 $dp(e-1, x-1)$ ；

鸡蛋未破碎：问题规模变为 $dp(e, f-x)$ 。

由于需要保证最坏情况下也能够找到临界楼层，因此需要取两种情况中的较大值。于是动态规划状态转移方程为：

$$dp(e, f) = \min_{1 \leq x \leq f} (1 + \max(dp(e-1, x-1), dp(e, f-x)))$$

边界条件： $dp[e][0]=0$ ， $dp[e][1]=1$ ， $dp[1][f]=f$

1. 蛮力递归法

蛮力递归法通过枚举每一层楼作为当前实验楼层，并分别递归计算鸡蛋破碎与未破碎两种情况。最终取所有方案中的最小值作为答案。

该算法虽然实现简单，但由于会重复计算大量相同子问题，因此时间复杂度接近指数级。随着楼层数增加，运行时间会急剧增长。

核心伪代码

```
Function BruteForce(e, f)
if f == 0 or f == 1 return f
if e == 1 return f
ans = INF
```

```

for x = 1 to f
    broken = BruteForce(e-1, x-1)
    notBroken = BruteForce(e, f-x)
    ans = min(ans, 1 + max(broken, notBroken))
return ans

```

2. 记忆化搜索

记忆化搜索是在蛮力递归基础上的优化。其核心思想是利用缓存数组记录已经计算过的状态，避免重复递归计算。

由于每个状态只会被计算一次，因此时间复杂度显著降低。记忆化搜索的时间复杂度约为 $O(ef^2)$ 。

核心伪代码

```

Function MemoDFS(e, f)
if memo[e][f] exists return memo[e][f]
if f == 0 or f == 1 return f
if e == 1 return f
ans = INF
for x = 1 to f
    broken = MemoDFS(e-1, x-1)
    notBroken = MemoDFS(e, f-x)
    ans = min(ans, 1 + max(broken, notBroken))
memo[e][f] = ans
return ans

```

3. 二分优化

在记忆化搜索中，每个状态都需要枚举所有楼层，因此仍然存在较大的时间开销。观察状态转移方程可以发现：

鸡蛋破碎部分随着楼层增加而增大；

鸡蛋未破碎部分随着楼层增加而减小。

因此两部分具有单调性，可以通过二分查找寻找最优楼层，从而减少枚举次数。

二分优化后的时间复杂度约为 $O(ef \log f)$ 。

核心伪代码

```

Function BinaryDFS(e, f)
left = 1, right = f
while left <= right
    mid = (left + right) / 2
    broken = BinaryDFS(e-1, mid-1)

```

```

notBroken = BinaryDFS(e, f-mid)
ans = min(ans, 1 + max(broken, notBroken))
if broken < notBroken
    left = mid + 1
else
    right = mid - 1
return ans

```

4. 优化状态法

优化状态法重新定义动态规划状态。设 $dp(m,e)$ 表示在 m 次实验、 e 个鸡蛋条件下最多能够确定多少层楼。

若当前实验鸡蛋破碎，则能够确定下方楼层；若鸡蛋未破碎，则能够确定上方楼层。

因此状态转移方程为：

$$dp(m,e)=dp(m-1,e-1)+dp(m-1,e)+1$$

相比传统状态定义，该方法无需枚举所有楼层，因此效率最高。实验中该算法在超大规模数据下仍能快速完成计算。

核心伪代码

```

Function OptimizedState(e, f)
initialize dp array
moves = 0
while dp[e] < f
    moves++
    for eggs from e downto 1
        dp[eggs] = dp[eggs] + dp[eggs-1] + 1
return moves

```

四、算法测试结果及效率分析

实验首先利用小规模随机数据对四种算法进行正确性验证。

小数据正确性测试（四种算法对比）

=====

测试 1: e= 3, f= 2 | 蛮力= 2, 记忆化= 2, 二分= 2, 优化状态= 2
测试 2: e= 1, f= 5 | 蛮力= 5, 记忆化= 5, 二分= 5, 优化状态= 5
测试 3: e= 1, f= 4 | 蛮力= 4, 记忆化= 4, 二分= 4, 优化状态= 4
测试 4: e= 1, f= 2 | 蛮力= 2, 记忆化= 2, 二分= 2, 优化状态= 2
测试 5: e= 3, f= 9 | 蛮力= 4, 记忆化= 4, 二分= 4, 优化状态= 4
测试 6: e= 1, f=10 | 蛮力= 10, 记忆化= 10, 二分= 10, 优化状态= 10
测试 7: e= 2, f= 1 | 蛮力= 1, 记忆化= 1, 二分= 1, 优化状态= 1
测试 8: e= 1, f= 2 | 蛮力= 2, 记忆化= 2, 二分= 2, 优化状态= 2
测试 9: e= 1, f= 4 | 蛮力= 4, 记忆化= 4, 二分= 4, 优化状态= 4
测试10: e= 3, f=10 | 蛮力= 4, 记忆化= 4, 二分= 4, 优化状态= 4

验证项	结果
随机小规模测试组数	20
4 种方法结果一致组数	20
通过率	100%

结果表明，所有算法在相同输入下均得到一致结果，说明算法实现正确。

随后分别对不同算法进行大规模效率测试，并绘制运行时间与理论复杂度趋势图。

记忆化搜索图分析

记忆化搜索的实际运行时间随着楼层数增加呈明显上升趋势，尤其在楼层数从 400 增加到 800 后，运行时间增长较快。这与理论复杂度 $O(ef^2)$ 基本一致。在固定鸡蛋数 $e=3$ 的情况下，时间主要受楼层数 f 影响，由于每个状态都要枚举楼层，所以楼层数增加会导致计算量近似平方级增长。

记忆化搜索相比蛮力递归已经避免了大量重复计算，但它仍然需要对每个状态枚举所有可能的投掷楼层，因此当楼层数较大时，效率仍然存在明显瓶颈。

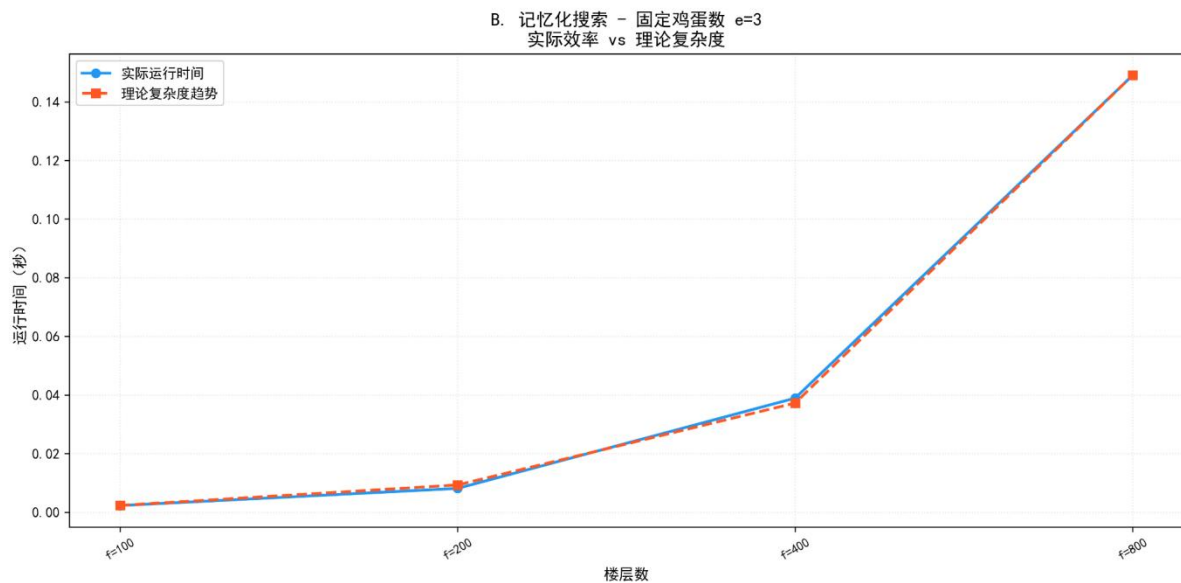


图 1 算法运行效率与理论复杂度趋势图

二分优化图分析

二分优化图中，实际运行时间随着楼层数增加而增长，但增长速度明显比记忆化搜索更平缓。其理论复杂度约为 $O(e f \log f)$ 。在固定鸡蛋数 $e=3$ 时，楼层数增加后，算法不再逐层枚举，而是利用二分查找寻找较优投掷楼层，因此计算量增长较慢。

从图中可以看出，二分优化的实际效率曲线与理论复杂度趋势较接近，说明利用“鸡蛋碎”和“鸡蛋不碎”两种情况的单调性能有效减少搜索范围。相比记忆化搜索，二分优化在中大规模数据下优势更加明显。

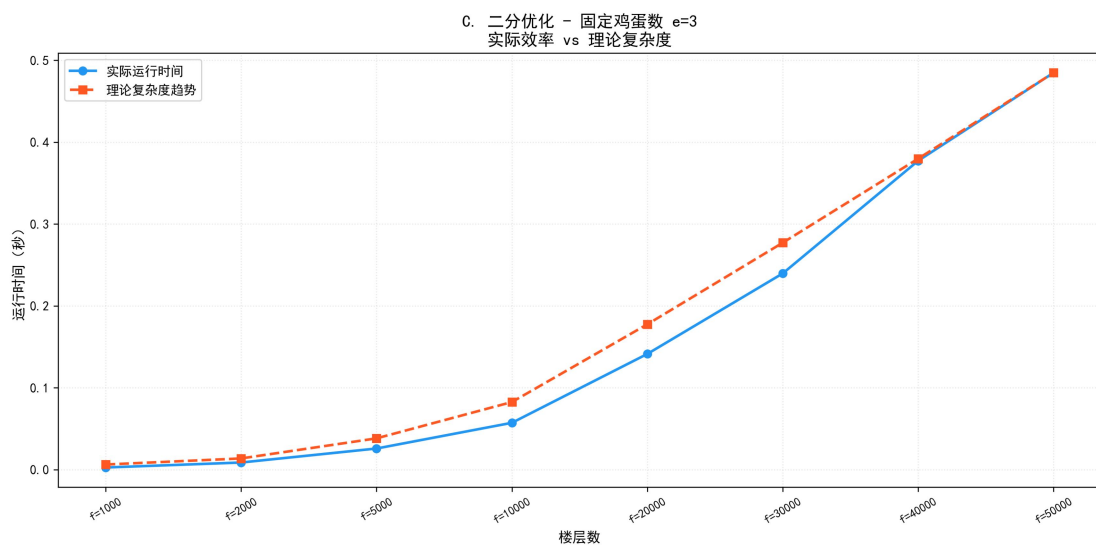


图 2 算法运行效率与理论复杂度趋势图

优化状态法图分析

优化状态法图中，即使楼层数增加到很大规模，实际运行时间仍然保持在极低水平，增长趋势非常缓慢。这说明优化状态法的效率明显高于前两种方法。

该算法的理论复杂度约为 $O(em)$ ，其中 m 表示最终需要的最少实验次数。由于 m 通常远小于楼层数 f ，所以该算法不需要围绕楼层数进行大量枚举。它通过重新定义状态，把问题从“给定楼层求次数”转化为“给定次数和鸡蛋数能覆盖多少楼层”，大幅降低了计算量。

从图中可以看出，优化状态法的实际运行时间与理论复杂度趋势基本一致，并且整体运行时间远低于记忆化搜索和二分优化，是本实验中效率最高的方法。

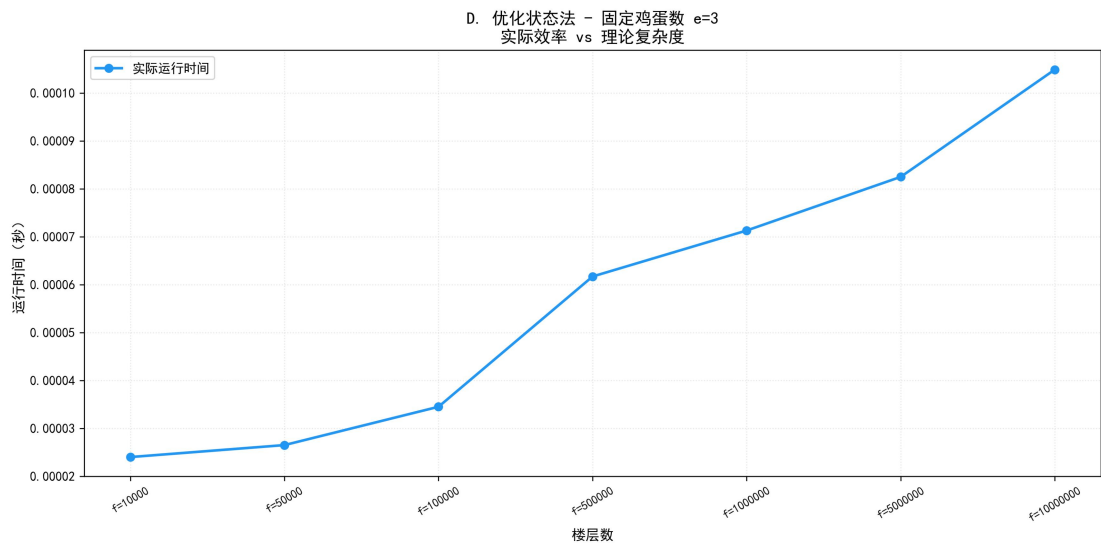
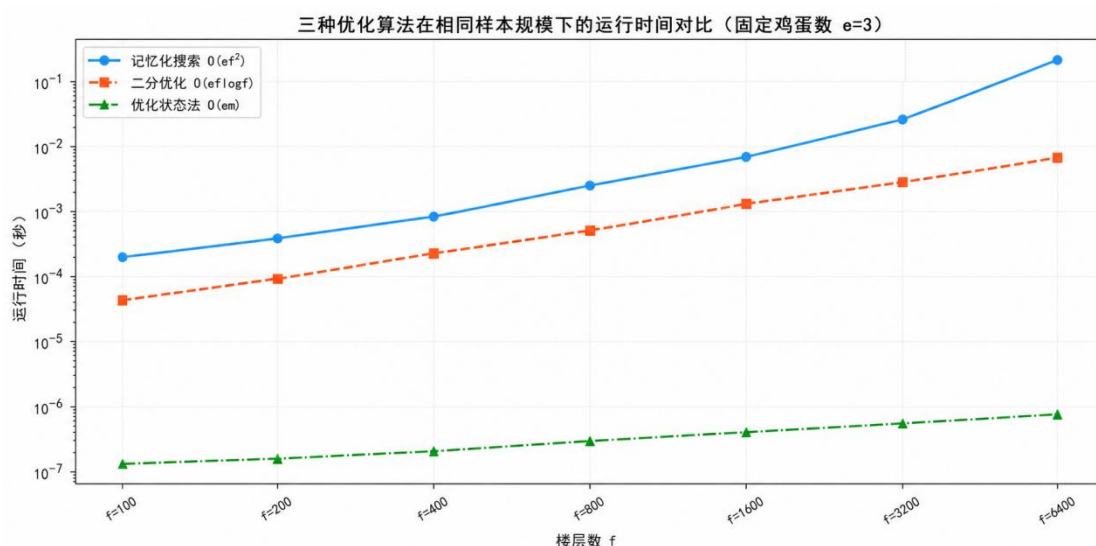


图 3 算法运行效率与理论复杂度趋势图

三种优化算法总体对比

三种优化算法的理论效率如下：

算法	理论时间复杂度	特点
记忆化搜索	$O(e^2f)$	避免重复递归，但仍需枚举楼层
二分优化	$O(e \log f)$	利用单调性减少枚举次数
优化状态法	$O(em)$	重新定义状态，效率最高



综合来看，三种算法体现了动态规划优化的递进过程。记忆化搜索解决了蛮力递归中的重复计算问题；二分优化进一步减少了每个状态中的楼层枚举；优化状态法则通过改变状态定义，从根本上降低了算法复杂度。实验结果表明，随着数据规模增大，算法效率差异越来越明显，优化状态法最适合处理大规模鸡蛋掉落问题。

五、经验总结

通过本次实验，进一步理解了动态规划中的最优子结构与重叠子问题思想。同时认识到，不同状态设计方式会对算法效率产生巨大影响。

从蛮力递归到记忆化搜索，再到二分优化与优化状态法，体现了算法逐步优化的过程，也加深了对时间复杂度分析与算法优化思想的理解。

指导教师批阅意见:

成绩评定：

指导教师签字：
年 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。